

A Voyage to the Kernel



Part 10

Day 9 | Segment 2.3

Algorithms in cryptography

The study of secret communication systems has lured people of all ages. And the old methods of encrypting messages are quite popular even in literature. But our interest is centred around two aspects—cryptography and cryptanalysis. Cryptography, in plain words, is concerned with the design of secret communications systems, while the latter studies the ways to compromise secret communications systems!

We all know that when a bank upgrades its systems to incorporate IT, it has to make sure that the methods of electronic funds transfer are just as secure as funds transferred by an armoured vehicle.

You might have seen the arithmetic and string-processing algorithms that people employ in this realm, which are what beginners are expected to study.

Cryptanalysis, for sure, can place an incredible strain on the available computational resources. That is why people consider this to be a very tedious process. To comprehend this, let's discuss a simple case of cryptography.

Let sender (S) send a message (called plaintext) to a particular receiver (R). 'S' converts his plaintext message to a secret form for transmission (which we may call the ciphertext) with the aid of a cryptographic algorithm (CA) and some defined key (K) parameters. 'CA' is the encryption method used here.

The whole procedure assumes some prior method of communication, as 'R' needs to know the parameters. The headache of the crypt analyst is that he needs to decipher the plaintext from the ciphertext without knowing the key parameters.

As we discussed, one of the simplest (and one of the oldest, too!) methods for encryption is the Caesar cipher method. Here, if a character in a particular place of the word is the N^{th} letter of the alphabet series, it is replaced by the $(N + K)^{\text{th}}$ letter in the series, where K is the parameter—an integer (Caesar used $K = 3$!).

```
CAESAR(CA,N,K)
for_all_characters
character (N+K)? character(N)
```

You can add more statements to fix bugs (say, if you're using English, you can specify what to do if $(N + K)$ exceeds 26).

Well, as said before, this method is very simple. Therefore, it's no big deal for the crypt analyst to crack the encrypted data. Things will become more complex if we use a general table to define the substitution and then use the same for the process. But here, too, our villain can try some tricks. He may choose the first character arbitrarily, say E (as E is the most frequent letter in English text). He may also choose not to go for certain diagrams such as QJ (as they never occur together in English).

You can develop the method further by using multiple look-up tables. Then, you will come across many interesting cases like the one when the key is as long as the plaintext ('one-time pad' case) and so on. It should be noted that if the message and key are encoded in binary, a more common scheme for position-by-position encryption is to use the "exclusive-or" function to encrypt the plaintext—"exclusive-or" it (bit by bit) with the key.